

Advanced Plant Image Classification with Ensemble Models

Group 25

İbrahim Ertürk 21902688
Sefa Baha Akdemir 21802079
Buse Yıldırım 21802290
Muhammed Gölcük 21802551

1 Introduction

The domain of plant species classification presents unique challenges due to the intricate and subtle variations among different species. Traditional classification methods, while effective to an extent, often struggle with the complexity and variability inherent in botanical datasets. This project aims to revolutionize plant species classification by synergizing the strengths of deep learning models and traditional machine learning techniques.

Our approach is novel in its dual-model ensemble strategy. The first ensemble model integrates the deep learning prowess of VGGNet with the interpretability and statistical robustness of Random Forest. This combination is meticulously designed to extract and utilize rich feature sets from images while simultaneously focusing on relevant image regions crucial for accurate species identification.

The second ensemble model implements the sophisticated feature extraction abilities of ResNet with the efficient gradient-boosting capabilities of LightGBM. This model is poised to offer a complementary yet distinctly powerful approach to plant classification. The ResNet architecture, renowned for its depth and ability to learn from large-scale image data, extracts nuanced features that are then effectively processed by LightGBM, known for its speed and performance in handling structured data.

Central to our methodology is the utilization of the "Pl@ntNet-300K" dataset, a comprehensive collection of over 300,000 images encompassing 1081 plant species. This dataset, with its vast diversity and volume, provides an ideal testing ground for our models. It mirrors real-world complexities and offers a rich resource for training and evaluating our innovative approaches.

2 Dataset

The cornerstone of any machine learning project is a robust and comprehensive dataset. In this project, we utilize the "Pl@ntNet-300K" dataset, a rich repository of over 306,293 images representing 1,081 distinct plant species. This dataset is particularly valuable for its diversity, which includes a wide range of plant images, from common species to rare and less-documented varieties. The dataset's extensive nature allows us to thoroughly train and test our models, ensuring they are capable of handling real-world variability and complexity in plant species classification.

2.1 Dataset Split

2.1.1 Strategy

The dataset was meticulously divided into training, validation, and testing subsets. We adopted a stratified sampling approach to maintain the representation of each species across these subsets. This method is crucial in addressing the dataset's inherent class imbalance and ensuring that rare species are adequately represented in each subset.

2.1.2 Allocation

The dataset allocation was as follows: 70% for training, 15% for validation, and 15% for testing. This distribution was chosen to provide a substantial amount of data for training the models while still retaining a significant portion for validation and testing, which is crucial for assessing the models' performance and generalizability.

2.2 Dataset Preprocessing

This part consists of image processing and data augmentation.

2.2.1 Image Processing

Images were resized from (570,570) to (224,224) to ensure consistency in input size for the models. Additionally, we applied normalization techniques to scale pixel values, aiding in the convergence of model training. Normalized pixel values have a mean of [0.485, 0.456, 0.406] and a standard deviation of [0.229, 0.224, 0.225]. The below figure shows an image before preprocessing and after preprocessing.



Figure 1: A sample image before preprocessing and after preprocessing

2.2.2 Data Augmentation

To enhance the robustness of our models against overfitting and to simulate a more diverse range of field conditions, we employed data augmentation techniques. These included random rotations, flipping, and scaling, which introduce variability and help the models generalize better to unseen data.

2.3 Detailed Analysis

2.3.1 Species Distribution Analysis

An in-depth analysis of species distribution within the dataset was performed. This analysis highlighted the presence of both common and rare species, guiding us in fine-tuning our models to handle this diversity effectively. We employed this method since we realized that some species had significantly lower examples of images for training.

2.3.2 Image Quality and Variability

We also assessed the quality and variability of the images. This assessment involved examining factors such as resolution, lighting conditions, and background noise. Understanding these aspects was crucial in developing preprocessing strategies and in adjusting model architectures to accommodate such variations.

2.3.3 Feature Distribution

A preliminary feature distribution analysis was conducted to understand the types of features present in the images and how they differ across species. This analysis informed the feature extraction strategies employed in our deep learning models.

3 Coding Environment Summary

The success of a machine learning project is heavily dependent on the choice of coding environment, which encompasses the programming languages, frameworks, and hardware utilized. In this project, we have carefully selected tools that are not only at the forefront of machine learning technology but also offer the robustness and flexibility needed for our complex models.

PyTorch, a leading deep learning framework, was selected for model development. PyTorch is renowned for its dynamic computation graph, which offers flexibility in model design and is particularly advantageous for complex architectures like ours. Its intuitive interface and extensive documentation have facilitated rapid prototyping and experimentation.

NVIDIA’s CUDA technology has been employed to leverage the power of GPUs for model training and inference. CUDA enables parallel processing, significantly speeding up computations necessary for handling large-scale image data and complex model architectures. This acceleration is crucial for iterative model tuning and experimentation, allowing for more efficient development cycles.

Our project utilizes fairly old GPUs with substantial memory and computing capabilities. We had GTX 1650, GTX 1050Ti, and we utilized Google Collab’s processing power. These GPUs are adept at processing the large batches of image data required for training our deep learning models, ensuring that the computational demands of both training and inference are met efficiently.

We used Visual Studio 2022 as our IDE since it was only a CUDA-supporting environment. We have implemented a robust version control system to manage our codebase. This system facilitates collaborative development, allowing multiple team members to work concurrently on different aspects of the project. It also ensures that our codebase remains organized and that changes are tracked systematically.

Collaborative tools such as Jupyter Notebooks and shared code repositories are employed to enhance teamwork and knowledge sharing. These tools allow for real-time collaboration, code sharing, and documentation, making it easier to integrate the work of different team members and maintain a cohesive development process.

4 Machine Learning Models

The essence of our project lies in the development and refinement of two innovative ensemble models, each combining the strengths of deep learning architectures with traditional machine learning algorithms. These models are designed to tackle the intricate task of plant species classification with high accuracy and interpretability.

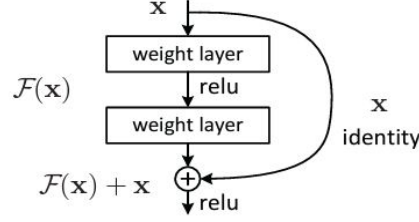


Figure 2: Residual Learning [1]

ResNet, known for its deep architecture and residual learning framework, is employed for its exceptional ability to extract complex features from images. We utilize variants of ResNet (ResNet-18 and ResNet-50) to capture a wide array of features at different levels of abstraction. The deep residual networks effectively mitigate the vanishing gradient problem, allowing for deeper network architectures without loss in performance.

```
t_model = models.resnet18(pretrained=True)
t_model = nn.Sequential(*list(resnet_model.children())[:-1]) # Remove the final fully connected layer
t_model.eval()
```

Figure 3: Resnet Implementation

The architecture of ResNet includes skip connections or shortcuts to jump over some layers. Typical configurations of ResNet used in our project include 18 and 50 layers. These configurations were chosen for their balance between computational efficiency and feature extraction capability.

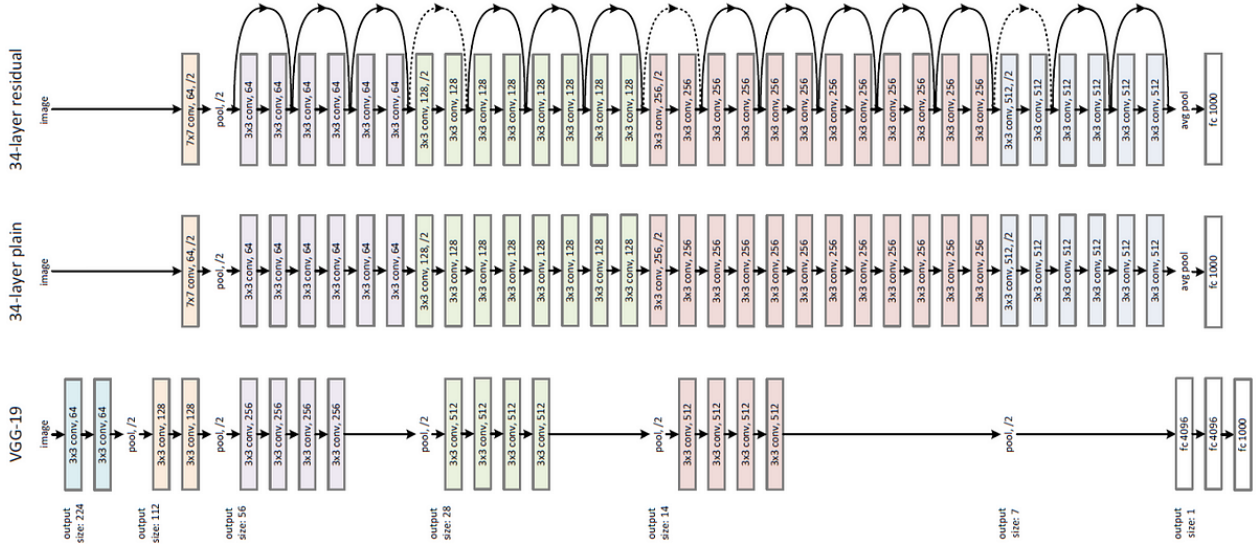


Figure 4: VGG vs Resnet Layer Comparison [1]

4.1 Model 1: ResNet18 + LightGBM

In the initial phase, we attempted to integrate LightGBM, a gradient-boosting framework, to complement the deep features extracted by ResNet. The integration involved removing the last layer of ResNet and fine-tuning LightGBM's hyperparameters, such as the number of leaves, minimum data in leaf, and learning rate, to optimize the model for our specific dataset. However, the accuracy achieved with this combination using ResNet18 was 13%. The confusion matrix of these two combinations is in the figure below. Since there are about 1000 classes, it was not possible to construct the confusion matrix for all classes. Instead, the top 20 classes that occurred in the dataset were used while constructing the confusion matrix.

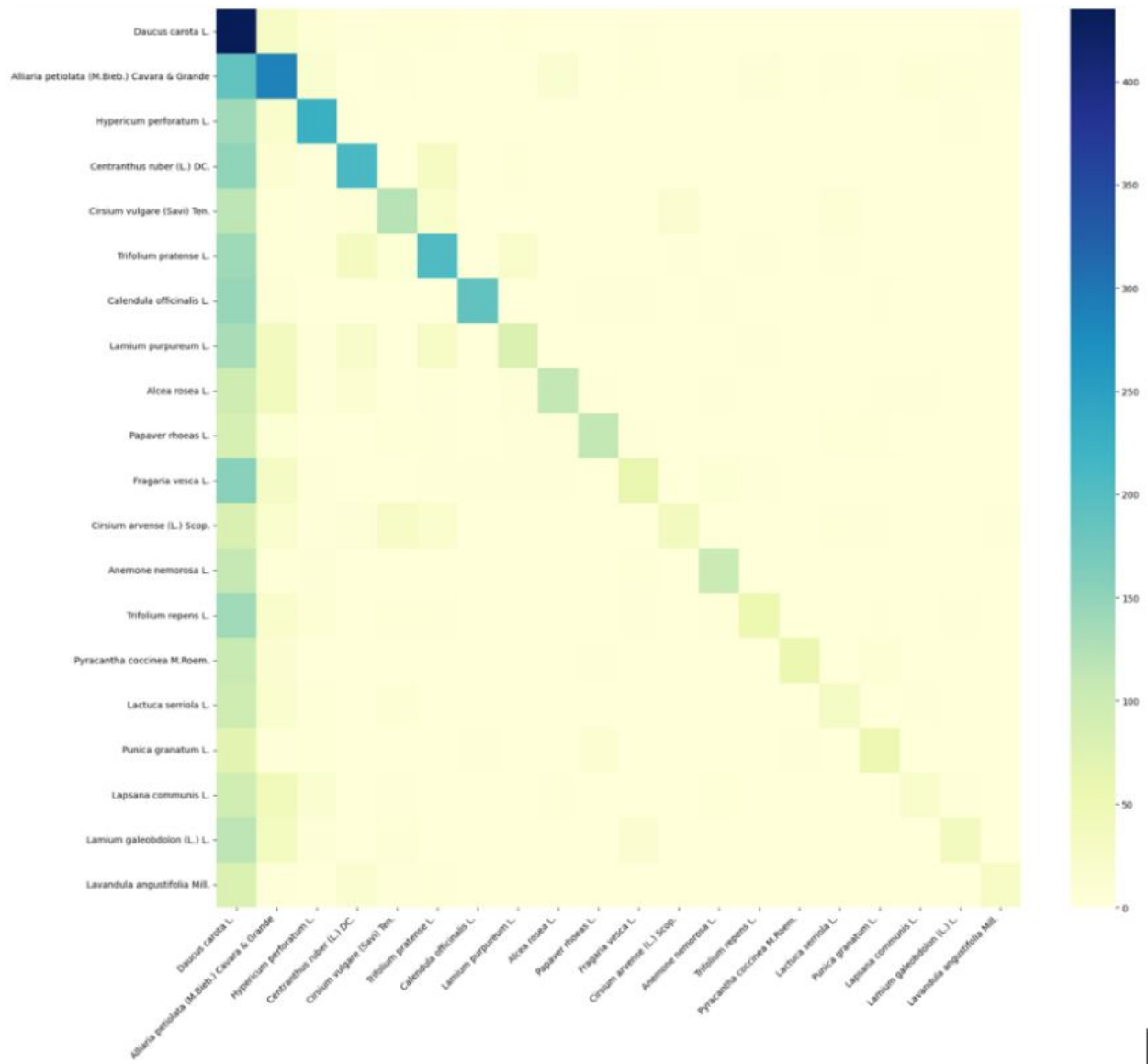


Figure 5: Confusion Matrix for the algorithm combining ResNet18 and LightGBM

4.2 Model 2: ResNet50 + LightGBM

Besides extracting features using ResNet18, we attempted combining ResNet50 with LightGBM, in which the accuracy was 16%. So, despite our efforts, the accuracy obtained did not meet our expectations. The confusion matrix of these two combinations with top-20 encountered classes are in the figure below.

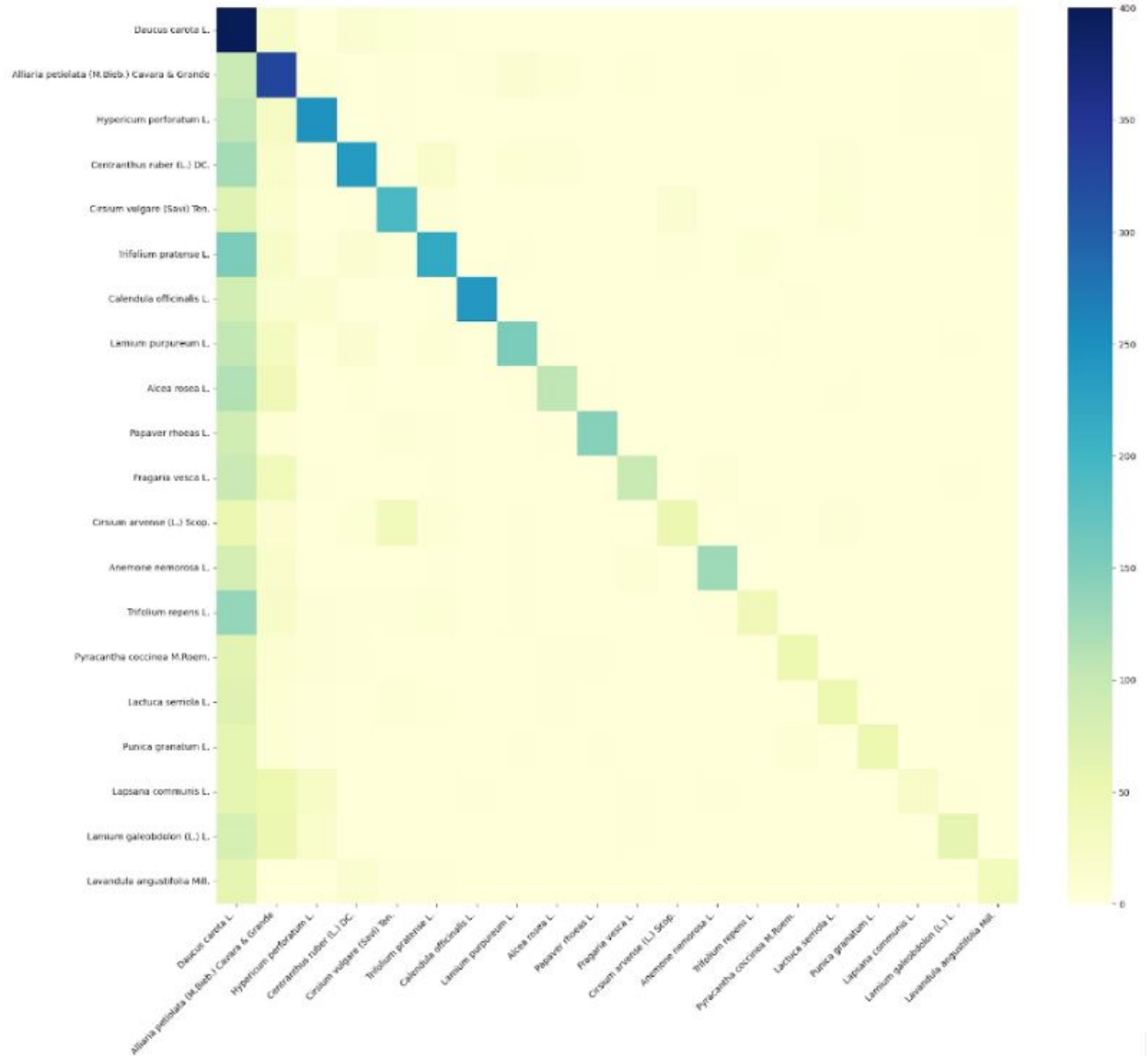


Figure 6: Confusion Matrix for the algorithm combining ResNet50 and LightGBM

4.3 Model 3: ResNet18 + Fully Connected Linear Layer

In response to the limitations observed using the first two model approaches, we explored an alternative ensemble approach by replacing LightGBM with a fully connected layer in the ResNet-18 architecture. This modification aimed to leverage the powerful feature extraction capabilities of ResNet while enhancing interpretability through a simpler and more transparent machine learning layer. Our experimentation revealed a significant improvement in accuracy with this modified architecture, which received 68% accuracy. The confusion matrix of this model is shown in figure below.

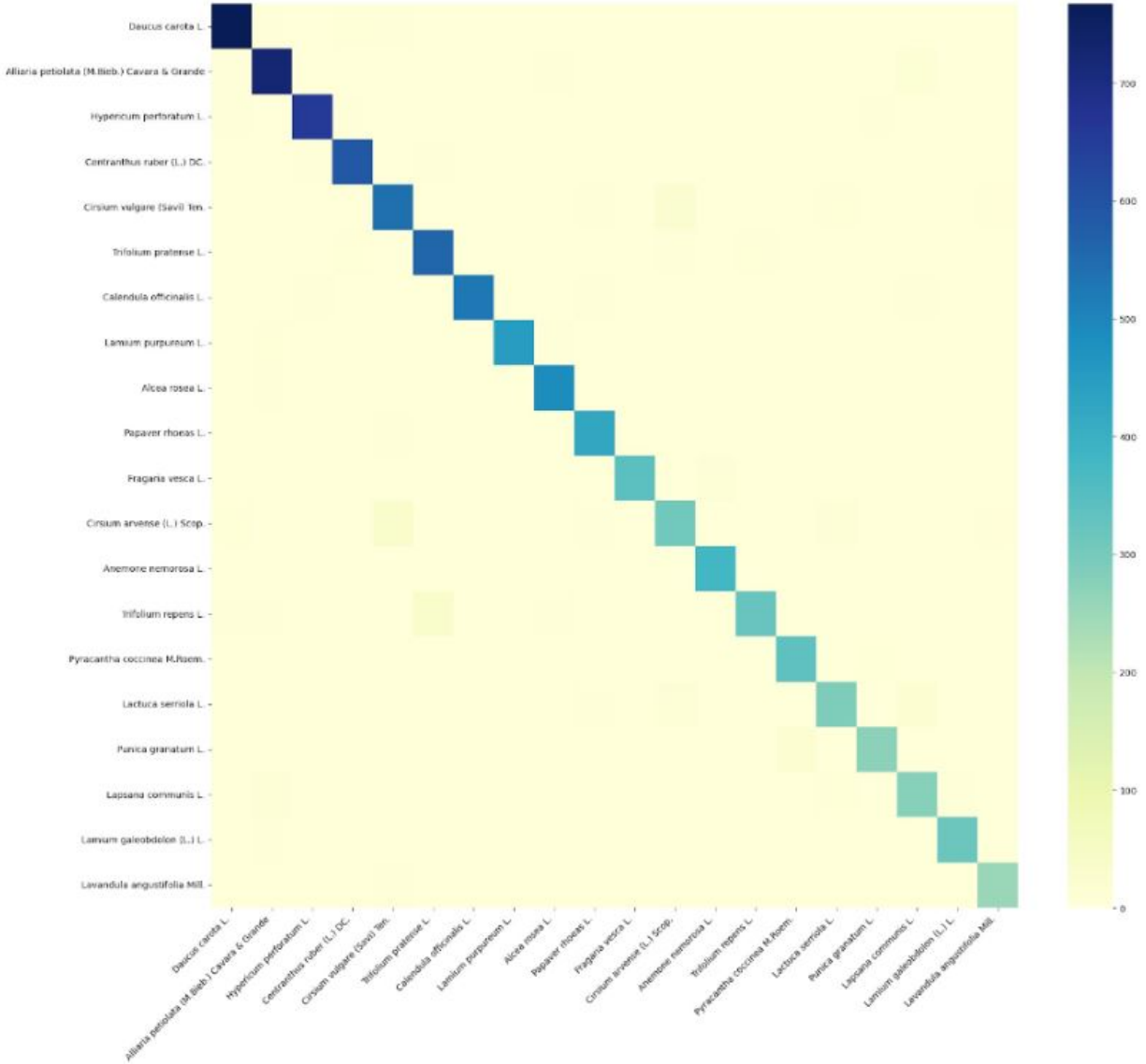


Figure 7: Confusion Matrix for a combination of ResNet18 and a fully connected layer

4.4 Model 4: ResNet152 + Fully Connected Linear Layer

To address limitations in earlier models, particularly with LightGBM integration, we replaced LightGBM with a fully connected linear layer in the ResNet-152 architecture. ResNet-152, chosen for its depth, enhanced the model's feature-capturing capabilities. The modification involved removing ResNet-152's last layer and adding a fully connected layer with tuned hyperparameters. The best hyperparameters were identified through a rigorous search, and the selected configuration was as follows:

- Learning Rate: 0.001
- Batch Size: 512

The model achieved a significant accuracy boost to 47 %, showing ResNet-152's effectiveness and interpretability with the added fully connected layer. The confusion matrix for this mode approach is shown below.

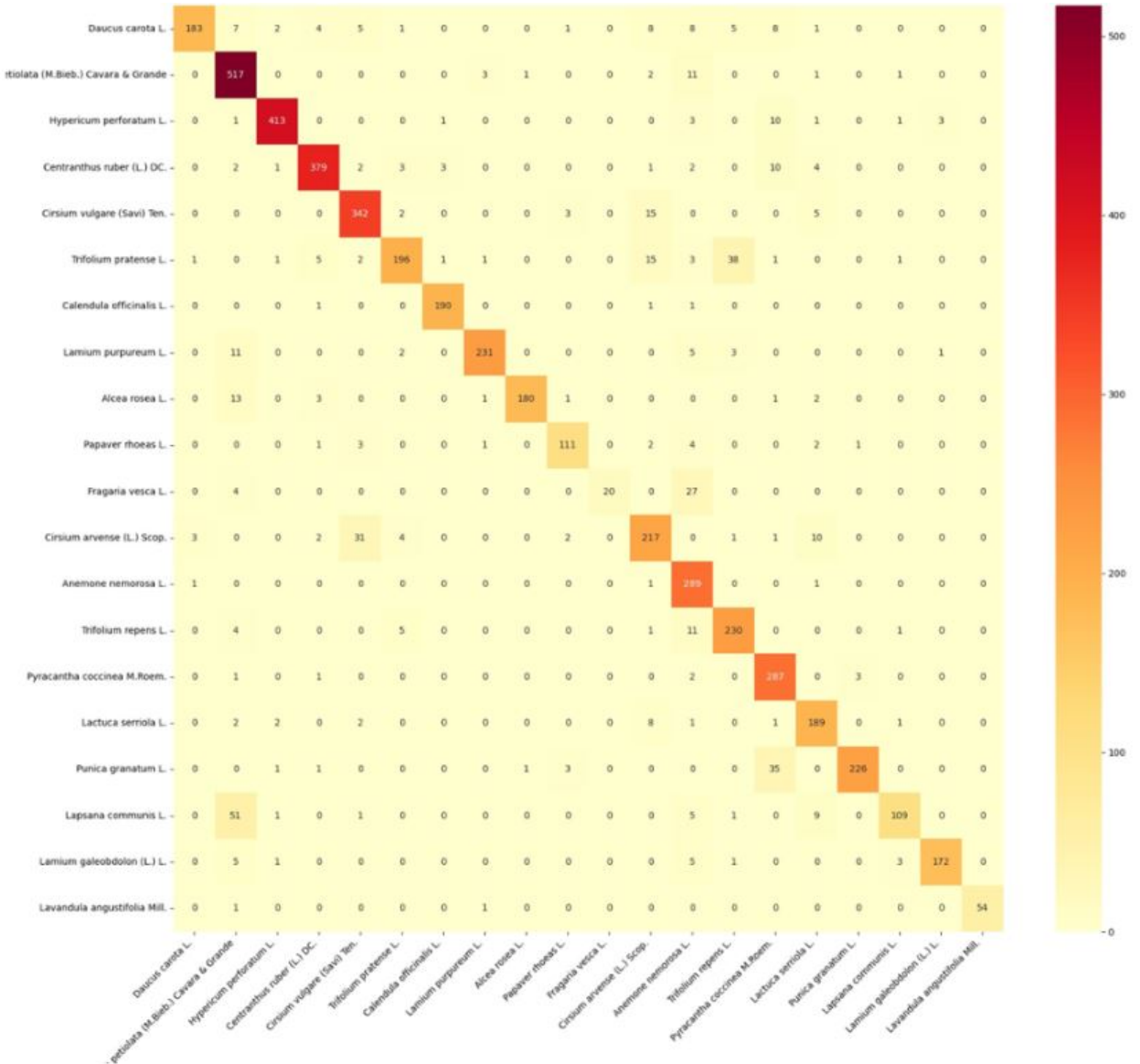


Figure 8: Confusion Matrix for a combination of ResNet152 and a fully connected layer

4.5 Model 5: VGG13

Our attempt to incorporate VGG13 into our ensemble strategy aimed to leverage its renowned depth and feature extraction capabilities for plant species classification. While VGG13 promised rich hierarchical features, the execution time proved to be a significant challenge, extending to approximately three months. This prolonged duration prompted a reevaluation of the feasibility and practicality of deploying such a model within our project constraints. Despite these challenges, the exploration of VGG13 highlighted the trade-off between feature richness and computational efficiency.

5 Validation and Evaluation

In machine learning, particularly in a domain as complex as plant species classification, rigorous validation and evaluation are paramount to ensure the reliability and generalizability of models. Our project employs a comprehensive suite of evaluation techniques to thoroughly assess the performance of our ensemble models.

We conducted rigorous validation and evaluation procedures for both models, including Cross-validation to assess the models’ performance, evaluation using set-valued metrics such as macro-average top-K accuracy, macro-average average-K accuracy, and mean bin accuracy, thorough hyperparameter tuning, and model optimization.

5.1 Cross-Validation

Cross-validation is a cornerstone of our validation strategy. We employed k-fold cross-validation, where the dataset was divided into k distinct subsets. Each subset, in turn, served as the test set while the remaining k-1 subsets were used for training. This method allowed us to utilize the entire dataset for both training and testing, providing a robust assessment of model performance. For our project, we chose a 5-fold cross-validation scheme, balancing computational efficiency with the reliability of the results.

5.2 Set-Valued Metrics

To gain a comprehensive understanding of our models’ performance, we used set-valued metrics such as macro-average top-K accuracy and macro-average average-K accuracy. These metrics are particularly useful in multi-class classification problems like ours.

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC’14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Figure 9: Error Comparison of different models [1]

5.2.1 Macro-Average Top-K Accuracy:

This metric calculates the accuracy of the model in predicting the top K classes for each sample and then averages these accuracies across all classes. It is crucial for our project since it provides insights into the model’s ability to identify the correct species among its top predictions, an important aspect in a scenario with a large number of classes.

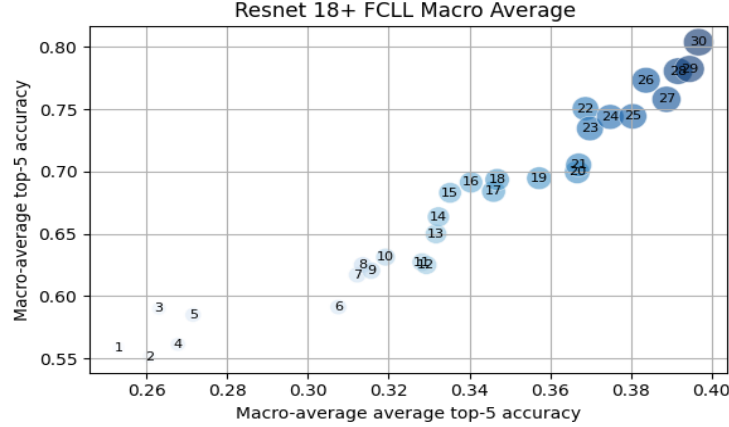


Figure 10: Macro Averages Result Plot of Resnet18+FCLL

5.2.2 Macro-Average Average-K Accuracy:

Differing slightly from top-K accuracy, this metric averages the accuracies obtained for each class and then computes the average across these averages. This approach ensures that all classes, regardless of their frequency in the dataset, contribute equally to the final score, which is vital for datasets with class imbalances like ours.

5.3 Hyperparameter Tuning and Model Optimization

Hyperparameter tuning is an integral part of our model optimization process. We utilized techniques like grid search and random search to systematically explore a wide range of hyperparameter combinations. For our deep learning models, parameters such as learning rate, batch size, and the number of epochs were meticulously tuned. Similarly, for our machine learning algorithms like a fully connected layer, parameters such as the batch size and learning rate were optimized.

This process was not only about finding the best hyperparameter values but also about understanding the interaction between different parameters and their impact on model performance. The insights gained from this exercise were invaluable in fine-tuning our models to achieve the best possible results.

5.4 Model Evaluation and Iterative Refinement

The final stage of our validation and evaluation process involved an iterative refinement of our models. Based on the results from cross-validation and set-valued metrics, we identified areas where our models could be improved. This iterative process involved tweaking model architectures, adjusting preprocessing steps, and even revisiting our feature engineering strategies.

For instance, if a model showed a tendency to overfit, we would experiment with adding dropout layers or increasing the regularization. If a model was underperforming in recognizing certain rare species, we might revisit our data augmentation strategies to provide a more balanced representation of these species.

5.5 Mean Bin Accuracy

This accuracy showed us how the accuracy changes with plant classes when 600 to 1081(all classes) included, we reached the highest accuracy. This data shows how the same plant species were not trained enough to recognize their classes. Also shows that an unknown species introduced will probably be flagged as false positive as another species 1081 species of plant we trained.

Classes with Images	Accuracy
0 – 10	5.15%
10 – 50	5.97%
50 – 300	8.76%
300 – 600	13.59%
600 – inf	33.38%

Table 1: Mean Bin Accuracy for Model 1

Classes with Images	Accuracy
0 – 10	5.46%
10 – 50	9.62%
50 – 300	11.84%
300 – 600	16.92%
600 – inf	37.15%

Table 2: Mean Bin Accuracy for Model 2

Classes with Images	Accuracy
0 – 10	17.80%
10 – 50	46.65%
50 – 300	69.86%
300 – 600	80.35%
600 – inf	87.10%

Table 3: Mean Bin Accuracy for Model 3

Classes with Images	Accuracy
0 – 10	21.38%
10 – 50	45.46%
50 – 300	54.24%
300 – 600	38.11%
600 – inf	36.53%

Table 4: Mean Bin Accuracy for Model 4

6 Workload Distribution

- İbrahim Ertürk: Implemented ResNet18+ LightGBM and VGG13.
- Sefa Baha Akdemir: Implemented ResNet50+ LightGBM and hyperparameter optimization.
- Buse Yıldırım: Implemented Resnet 152 +Fully Connected Linear Layer.Did Preprocessing.
- Muhammed Gölcük: Resnet18+Fully Connected Linear Layer, Implemented validation methods.

Everyone worked on the presentation for the demo and wrote the report together. We worked on GitHub and Google Colab. Training for data was completed with İbrahim Ertürk and Muhammed Gölcük’s computers (GTX1650 and GTX1050ti) and on Google Colab, totaling 15 hours.

7 Conclusion

In conclusion, our project set out to redefine plant species classification, employing a strategic ensemble of deep learning and traditional machine learning models. Among the models explored, the combination of ResNet18 and a fully connected linear layer emerged as the most successful, achieving a remarkable accuracy of 68%. This architecture effectively harnessed ResNet’s feature extraction prowess while enhancing interpretability.

While the integration of VGG13 posed computational challenges, the project illuminated the delicate balance between feature richness and execution efficiency. Rigorous validation processes, including cross-validation and set-valued metrics, ensured the robustness of our models. As a collaborative team, we navigated challenges, emphasizing thoughtful model selection, validation, and iterative refinement. The ResNet18 with a linear layer stands out as the winner, contributing valuable insights to the field of plant species classification.

8 References

1. K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. doi:10.1109/cvpr.2016.90